

Benchmarking CometBFT with Asymmetric Grid Quorum Systems

Michael Senn
michael.senn@unibe.ch
University of Bern
Institute of Computer Science
Bern, Switzerland

Christian Cachin
christian.cachin@unibe.ch
University of Bern
Institute of Computer Science
Bern, Switzerland

A Artifact Appendix

A.1 Abstract

This artifact accompanies a paper¹ introducing *asymmetric grid quorum systems*, a quorum system construction which is based on differences of real-world processes such as their operating system, or physical location.

The artifact implements a benchmark comparing two versions of CometBFT, a widely-used state-machine replication software. One version, titled *baseline*, is an unmodified recent stable version of CometBFT. The other, titled *grid*, is a version of CometBFT modified to work with these grid quorums instead.

The artifact consists of a pipeline to assist with deploying CometBFT on a set of machines, a benchmarking client to apply a workload and collect performance data, and a data analysis and visualization toolchain to prepare figures based on this data. Results show that adding support for these quorum systems can be done with relatively low effort, and that doing so has no adverse effect on the performance of a real-world system such as CometBFT.

The artifact is provided as a set of public Github repositories, as well as as a point-in-time snapshot on Zenodo. All code is licensed under an Apache 2.0 license.

A.2 Description & Requirements

For organizational reasons, our artifact is split into multiple components. These are:

tenderload Main benchmarking pipeline, containing code to automatically provision machines, run the benchmark, collect data, and analyze results. Also contains the raw data used to produce the results of the paper.

tendergrid A fork of CometBFT, with added support for asymmetric quorum systems.

tendercounter A replicated application running on top of CometBFT.

asymmetric-quorums A Go library providing methods to interface with asymmetric quorum systems.

Each component contains a `README.md` explaining how to use the tools contained within. To get started with the artifact as a whole, we recommend consulting the top-level readme of the `tenderload` repository. It contains thorough step-by-step instructions on how to reproduce the results of the paper.

A.2.1 Security, privacy, and ethical concerns. There is no direct risk with executing any of the artifacts, other than the risk inherent

with running software. None of the artifact contains functionality purposefully made to be harmful or destructive.

A.2.2 How to access. We provide the artifact as a set of public repositories on GitHub^{2,3,4,5} as well as a point-in-time snapshot on Zenodo⁶.

Most of these repositories are labelled with a git tag `ccs26`, marking the version used to produce the paper's results. The `tendergrid` repository instead contains two tags, `ccs26-baseline` and `ccs26-grid`, marking the version of CometBFT used for baseline measurements, and those with support for asymmetric grid quorums respectively. The snapshot on Zenodo instead contains a single zip archive for each repository and tag.

A.2.3 Hardware dependencies. To make use of the full artifact, a set of machines on which to host a CometBFT deployment is required. These can be hosted in a cloud, on a local hypervisor, or even as a set of VMs on a personal computer. To reproduce the results of the paper fully, you must be able to host 49 VMs, each with 4 CPU cores and 4 GiB of RAM, connected by a high-speed network, without causing resource contention among the machines. The machines themselves do not require specific hardware, but should be based on the x86 architecture to make use of pre-compiled binaries. Other architectures will work as well, but may require building parts of the artifact for that architecture, first.

A.2.4 Software dependencies. Depending on the parts of the artifact one wants to use, different software is required. Detailed instructions are provided in the main readme of the `tenderload` repository. Broadly speaking, these are:

- Debian 12, as operating system of the benchmark VMs.
- Ansible, to make use of the deployment pipeline
- Docker, to run the distributed application on the benchmarking machines and, optionally, build containers. We provide pre-built containers for x86.
- Optionally Go, to build the benchmarking client. We provide a pre-built binary for x86.
- Python, to make use of the data-analysis and visualization pipeline.

A.2.5 Benchmarks. No specific datasets are required. The benchmark pipeline is able to generate its own datasets for evaluation. Further, the raw data used for the paper's results are included as

¹<https://doi.org/10.1145/3830454.3832646>

²<https://github.com/cryptobern/tenderload>

³<https://github.com/cryptobern/tendergrid>

⁴<https://github.com/cryptobern/tendercounter>

⁵<https://github.com/cryptobern/asymmetric-quorums>

⁶<https://doi.org/10.5281/zenodo.20763187>

part of the artifact, if one wishes to e.g. only use the data analysis part of the artifact on existing data.

A.3 Set Up

For detailed setup instructions, consult the main readme of the tenderload repository.

A.3.1 Basic test. As a basic functionality test not involving large-scale deployments, consider checking whether the deployment pipeline works for a small network of e.g. $n = 4$ VMs. Follow the instructions in the main readme of the tenderload repository to deploy the baseline version of CometBFT. Once the deployment is through, check the log output of the CometBFT containers to ensure that it is producing blocks.

At that point, consider performing a benchmark with e.g. invocation rates between 10 and 100, and visualize the resulting data. For this choice of parameters, expect to see a 1:1 relation between invocation rate and throughput.

A.4 Evaluation Workflow

A.4.1 Major claims.

- (C1): Adding support for asymmetric grid quorums to CometBFT does not meaningfully impact its throughput, compared to an equivalent benchmark for an unmodified version of CometBFT, for LAN deployments with $n = 28$ and $n = 49$ machines. This is proven by experiments (E1) and (E2) described in Sections 7 and 8 of our paper. Results of the experiment are reported in Section 8, and visualized in Figure 5 and Table 2.

A.4.2 Experiments. All instructions below, when referring to sections, refer to sections within the ‘Running the benchmark yourself’ Chapter of the main README.md in the tenderload repository.

- (E1): [Baseline] [8 person-hours (mostly wait-time) + 4 compute-weeks + 2 TiB disk]:

Preparation: Prepare a deployment of $n = 28$ (respectively $n = 49$) CometBFT instances using the *baseline* version of CometBFT, by following Sections ‘Requirements’ through ‘Start CometBFT network’.

This should lead to a working deployment of CometBFT, which produces blocks at a rate of one per second.

Execution: Run a benchmark by following Sections ‘Compiling and deploying the benchmark client’ and ‘Running the benchmark client’.

After termination of the benchmark client, this should have produced a set of raw benchmark data as a set of CSV and JSON files on the host on which the client ran.

Results: Collect results and plot them by following Sections ‘Downloading benchmark results’ and ‘Plotting results’.

This should lead to a set of PNG images, containing invocation-throughput plots.

- (E2): [Grid] [10 person-hours (mostly wait-time) + 5 compute-weeks + 2 TiB disk]:

Preparation: Equivalent to *Preparation* of (E1), but deploy the *grid* version of CometBFT instead. Be sure *not* to skip

the ‘Create sample asymmetric grid quorum system’ subsection.

Execution: Equivalent to *Execution* of (E1). Be sure to read and apply the instructions in the ‘Salvaging half-successful benchmark runs’ Subsection.

Results: Equivalent to *Results* of (E1).

A.5 Notes on Reusability

The benchmarking pipeline is flexible enough that parameters such as the size of the system can be easily changed. This is documented in some detail within the tenderload repository, both within readme files as well as with comments within the Ansible scripts.

More generally, the deployment pipeline could be used to deploy a fully-working CometBFT setup, including a replicated application (so-called ‘ABCI app’) for purposes other than our benchmark.

Lastly, the `asymmetric-quorums` Go library could be used in applications operating with more advanced quorum structures – both asymmetric as well as symmetric – than the traditional threshold quorums.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.